Ejercicios con Persistencia de Datos.

Ejercicio 1 — Agenda de contactos

Crear agenda.db y una tabla contactos:

```
id              INTEGER PRIMARY KEY AUTOINCREMENT
nombre          TEXT NOT NULL
telefono        TEXT NOT NULL
email           TEXT
```

El programa debe mostrar un menú:

1. Agregar contacto
2. Listar contactos
3. Buscar contacto por ID
4. Modificar contacto
5. Eliminar contacto
0. Salir

En la modificación, pedir el ID y permitir modificar nombre, teléfono y email.

Ejercicio 2 — Productos de un comercio

Crear comercio.db y una tabla productos:

```
id              INTEGER PRIMARY KEY AUTOINCREMENT
descripcion     TEXT NOT NULL
precio          REAL NOT NULL
stock           INTEGER NOT NULL
```

Implementar:

1. Agregar producto
2. Listar productos
3. Buscar producto por ID
4. Modificar precio y stock
5. Eliminar producto
6. Mostrar productos con stock menor a una cantidad ingresada
0. Salir

Consulta con condición:

```
SELECT *
FROM productos
WHERE stock < ?
```

Ejercicio 3 — Registro de alumnos

Crear escuela.db con:

```
alumnos
------------------------------
id          INTEGER PRIMARY KEY AUTOINCREMENT
dni         TEXT NOT NULL UNIQUE
apellido    TEXT NOT NULL
nombre      TEXT NOT NULL
curso       TEXT NOT NULL
```

Implementar el ABMC completo y agregar:

6. Buscar alumno por DNI

7. Listar alumnos de un curso

Consultas mas complejas:

```
cursor.execute("""
    SELECT *
    FROM alumnos
    WHERE dni = ?
""", (dni,))
```

y:

```
cursor.execute("""
    SELECT *
    FROM alumnos
    WHERE curso = ?
""", (curso,))
```

Ejercicio 4 — Biblioteca

Crear biblioteca.db con una tabla:

```
libros
------------------------------
id              INTEGER PRIMARY KEY AUTOINCREMENT
titulo          TEXT NOT NULL
autor           TEXT NOT NULL
anio            INTEGER
disponible      INTEGER NOT NULL
```

El campo disponible puede manejarse inicialmente como 1 y 0.

Además del ABMC:

6. Mostrar solamente libros disponibles

7. Buscar libros por autor

8. Marcar un libro como prestado

9. Marcar un libro como disponible

Las dos últimas opciones permiten practicar UPDATE sin modificar todo el registro:

```
UPDATE libros
SET disponible = 0
WHERE id = ?
```

Ejercicio 5 — Veterinaria

```
mascotas
------------------------------
id              INTEGER PRIMARY KEY AUTOINCREMENT
nombre          TEXT NOT NULL
especie         TEXT NOT NULL
raza            TEXT
edad            INTEGER
propietario     TEXT NOT NULL
telefono        TEXT
```

ABMC completo y consultas:

6. Buscar mascotas por propietario

7. Mostrar mascotas de una especie

8. Mostrar mascotas mayores a una edad ingresada

Caso Resuelto. Agenda

Codigo de acceso a datos.

```
import sqlite3

conexion = sqlite3.connect("agenda.db")
cursor = conexion.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS contactos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT NOT NULL,
        telefono TEXT NOT NULL,
        email TEXT
    )
""")
```

```
conexion.commit()


def agregar_contacto():
    nombre = input("Nombre: ")
    telefono = input("Teléfono: ")
    email = input("Email: ")

    cursor.execute("""
        INSERT INTO contactos (nombre, telefono, email)
        VALUES (?, ?, ?)
    """, (nombre, telefono, email))

    conexion.commit()


def listar_contactos():
    cursor.execute("""
        SELECT id, nombre, telefono, email
        FROM contactos
    """)

    contactos = cursor.fetchall()

    for contacto in contactos:
        print(contacto)

while True:
    print("\nAGENDA")
    print("1 - Agregar")
    print("2 - Listar")
    print("3 - Buscar")
    print("4 - Modificar")
    print("5 - Eliminar")
    print("0 - Salir")

    opcion = input("Opción: ")

    if opcion == "1":
        agregar_contacto()

    elif opcion == "2":
        listar_contactos()

    # ...

    elif opcion == "0":
        break

conexion.close()
```